

Práctica 2: Interfaz gráfica para el simulador de ecosistema

Objetivo: Diseño orientado a objetos, Modelo-Vista-Controlador, interfaces gráficas de usuario con Swing.

Fecha de entrega: 20 de Abril 2026, 09:00h

Control de Copias

Durante el curso se realizará control de copias de todas las prácticas, comparando las entregas de todos los grupos de TP2. Se considera copia la reproducción total o parcial del código de otros alumnos o cualquier código extraído de Internet o de cualquier otra fuente, salvo aquellas autorizadas explícitamente por el profesor. En caso de detección de copia, la calificación en la convocatoria de TP2 en la que se haya detectado la copia será 0.

Si decides almacenar tu código en un repositorio remoto, por ejemplo en un sistema de control de versiones gratuito con vistas a facilitar la colaboración con tu compañero de laboratorio, asegúrate de que tu código no esté al alcance de los motores de búsqueda. Si alguien que no sea el profesor de tu asignatura, por ejemplo un empleador de una academia privada, te pide que facilites tu código, debes negarte.

Instrucciones Generales

Las siguientes instrucciones son **estrictas**, es decir, **debes seguirlas obligatoriamente**.

1. Lee el enunciado **completo** de la práctica antes de empezar.
2. Si estás usando un repositorio git para tener tu práctica bajo el control de versiones, te aconsejo que pongas un *tag* a tu solución de la práctica 1 (puedes hacer esto usando el comando `git tag`) y realizar la práctica 2 en una rama nueva. Si no estás usando control de versiones, haz una copia de seguridad de la práctica 1 en un lugar seguro.
3. Echa un vistazo a las clases nuevas que os proporcionamos en el paquete `simulator.view`.
4. Es necesario usar exactamente la misma estructura de paquetes y los mismos nombres de clases que aparecen en el enunciado.
5. No está permitido el uso de ninguna herramienta para la generación automática de interfaces gráficas de usuario.
6. Echa un vistazo a los recursos que os proporcionamos (en `src/main/resources`):
 1. En `extra`, hay unos ejemplos de `JTable` y `JDialog`.
 2. Dentro de `icons` están los iconos que vamos a usar en la ventana de nuestro simulador.
7. No escribas errores con `System.out` ni con `printStackTrace()` de la excepción, todos los errores hay que mostrarlos usando

`ViewUtils.showErrorMsg`.

8. Cuando entregues la práctica sube un fichero con el nombre **src.zip** que incluya solo la carpeta **src**. No está permitido llamarlo con otro nombre ni usar **7zip**, **rar**, etc. Si usas iconos adicionales, se puede incluir la carpeta **src/main/resources/icons** mientras que el tamaño total del **zip** no supera los **100k**.

Descripción general de la interfaz Gráfica del Simulador

En esta práctica vas a desarrollar una interfaz gráfica de usuario (GUI) para el simulador de ecosistema siguiendo el patrón de diseño Modelo-Vista-Controlador (MVC). En el apartado Figuras puedes ver la GUI que hay que construir. Está compuesta por una ventana principal que contiene cuatro componentes: (1) un panel de control para interactuar con el simulador; (2) una tabla que muestra información sobre las especies y su estado; (3) una tabla que muestra el estado de todas las regiones; y (4) una barra de estado en la que aparece más información, que detallaremos después.

Además, incluye un diálogo que permite cambiar las regiones, y una ventana (que se abre de forma separada) para dibujar el estado de la simulación (similar al visor que usaste en la primera práctica).

Ver demo.mp4

Cambios en el Modelo y el Controlador

Esta sección describe los cambios que hay que hacer en el modelo y el controlador para usar el patrón de diseño MVC y añadir alguna funcionalidad extra.

Método `reset` en la clase `Simulator`

Añade el siguiente método a la clase `Simulator` (si es que no lo tienes ya):

```
public void reset(int cols, int rows, int width, int height): vacía la lista de animales (o crea una nueva), crea un nuevo RegionManager con tamaño adecuado, y pone el tiempo a 0.0.
```

Método `toString()` en las clases de regiones

Añade un método `toString()` a todas las clases que extienden a la clase `Region` que devuelva una pequeña descripción correspondiente, por ejemplo:

- `DefaultRegion`
- `DynamicRegion`

Esta información se usará en la GUI para mostrar la descripción de una región.

El método `fillInData` de los builders

Completa el método `fillInData` en todos los builders para que rellene información correspondiente (por lo menos hacerlo para los builders de regiones, el resto no lo vamos a usar de momento). Por ejemplo, `getInfo()` de los builders de regiones tendrá que devolver los siguientes JSON:

```
{
  "type": "default",
  "desc": "Infinite food supply",
  "data": {}
}

{
  "type": "dynamic",
  "desc": "Dynamic food supply",
  "data": {
    "factor": "food increase factor (optional, default 2.0)",
    "food": "initial amount of food (optional, default 100.0)"
  }
}
```

Puedes hacer lo mismo con los builders de los animales (aunque no sea necesario para esta práctica).

Iterador de regiones en la clase `RegionManager`

Nuestro objetivo es dar acceso a las regiones desde fuera del modelo, de tal manera que no se pueda alterar sus estados.

Empezamos con la modificación de la interfaz `RegionInfo` para permitir el acceso a la lista de animales como `List<AnimalInfo>` en lugar de `List<Animal>`:

```
public interface RegionInfo extends JSONable {
    public List<AnimalInfo> getAnimalsInfo();
}
```

En la clase `Region` el método correspondiente será:

```
public List<AnimalInfo> getAnimalsInfo() {
    return new ArrayList<>(animals); // se puede usar Collections.unmodifiableList(animals);
}
```

Las dos opciones en la línea del `return` son válidas, la primera es segura para programación concurrente mientras la segunda no (esto es importante para la tercera práctica).

[!Note] Ejercicio muy importante (no para entregar, simplemente para entender): explicar por qué `return animals` no compila, mientras las opciones de arriba sí compilan.

Ahora vamos a modificar la clase `RegionManager` para que tenga un iterador que permite recorrer sobre las regiones.

[!Important] está prohibido añadir un método `getRegion(int row, int col)` para consultar la región en la posición `(row,col)` desde fuera, es obligatorio hacerlo con un iterador para practicar los iteradores.

Empezamos con la modificación de la interface `MapInfo` para incluir un record de información sobre las regiones e implementar la interfaz `Iterable`:

```
public interface MapInfo extends JSONable, Iterable<MapInfo.RegionData> {
    public record RegionData(int row, int col, RegionInfo r) {
    }
    // el resto de la interfaz es como antes
}
```

El registro `RegionData` simplemente incluye la posición de la región y la región como `RegionInfo` en lugar de como `Region`, para asegurarnos que no se altera su estado desde fuera.

Ahora implementa un iterador correspondiente en la clase `RegionManager` que recorra la matriz de regiones (por filas, de izquierda a derecha) y para cada región devuelve una instancia correspondiente de `RegionData`.

La interfaz `EcoSysObserver`

Los observers implementan la siguiente interfaz, que incluye varios tipos de notificaciones (colócala en el paquete `simulator.model`):

```
public interface EcoSysObserver {
    void onRegister(double time, MapInfo map, List<AnimalInfo> animals);
    void onReset(double time, MapInfo map, List<AnimalInfo> animals);
    void onAnimalAdded(double time, MapInfo map, List<AnimalInfo> animals, AnimalInfo a);
    void onRegionSet(int row, int col, MapInfo map, RegionInfo r);
    void onAdvance(double time, MapInfo map, List<AnimalInfo> animals, double dt);
}
```

Los nombres de los métodos dan información sobre el significado de los eventos que notifican. En cuanto a los parámetros: `map` es el gestor de regiones; `animals` es la lista de animales; `a` es un animal, `r` es una región, `time` es el tiempo actual de la simulación y `dt` es el `delta-time` usado en el paso de simulación correspondiente. Nótese que usamos los tipos `MapInfo`, `AnimalInfo` y `RegionInfo` en lugar de `Animal`, `RegionManager` y `Region` para no permitir alterar el estado de los objetos correspondientes desde fuera de la simulación.

Modifica la clase `Simulator` para que implemente `Observable<EcoSysObserver>` donde la interfaz `Observable<T>` está definida como:

```
public interface Observable<T> {
```

```

    void addObserver(T o);
    void removeObserver(T o);
}

```

Añade a la clase `Simulator` una lista de observadores, inicialmente vacía, y añade los siguientes métodos para registrar/eliminar observadores:

1. `public void addObserver(EcoSysObserver o)`: añade el observador `o` a la lista de observadores, si es no está ya en ella.
2. `public void removeObserver(EcoSysObserver o)`: elimina el observador `o` de la lista de observadores.

Envío de notificaciones

Modifica la clase `Simulator` para enviar notificaciones como se describe a continuación:

1. Al final del método `addObserver` envía una notificación `onRegister` **solo al observador que se acaba de registrar**, para pasarle el estado actual del simulador.
2. Al final del método `reset`, envía una notificación `onReset` a **todos los observadores**.
3. Al final del método `addAnimal` envía una notificación `onAnimalAdded` a **todos los observadores**.
4. Al final del método `setRegion` envía una notificación `onRegionSet` a **todos los observadores**.
5. Al final del método `advance` envía una notificación `onAdvance` a **todos los observadores**.

Nótese que la lista de animales hay que pasarla como `List<AnimalInfo>` a los observadores. Esto se puede hacer usando `new ArrayList<>(animals)` or `Collections.unmodifiableList(animals)`, la diferencia entre las dos formas la explicada anteriormente. Por ejemplo, la notificación de `advance` se puede hacer usando el siguiente método:

```

private void notifyOnAdvance(double dt) {
    List<AnimalInfo> animals = new ArrayList<>(animals);
    // para cada observador o, invocar o.onAdvance(time, regionMgr, animals, dt)
}

```

Cambios en la clase `Controller`

La clase `Controller` tiene que ser extendida con funcionalidad adicional (para evitar pasar el simulador a la GUI) como sigue:

1. `public void reset(int cols, int rows, int width, int height)`: llama a `reset` del simulador.
2. `public void setRegions(JSONObject rs)`: suponiendo que `rs` es una estructura JSON que incluye la clave “regions” (como en la primera prác-

- tica), modifica las regiones correspondientes usando `setRegion` del simulador. Hay que hacer refactorización del código del `loadData` para que no haya duplicación de código (porque `loadData` ya hacía algo parecido).
3. `public void advance(double dt)`: llama a `advance` del simulador.
 4. `public void addObserver(EcoSysObserver o)`: llama a `addObserver` del simulador.
 5. `public void removeObserver(EcoSysObserver o)`: llama a `removeObserver` del simulador.

Factorías y delta-time públicos en la clase Main

En la clase `Main`, hacer los atributos que corresponden a las factorías y el delta-time públicos porque se van a usar desde la GUI.

La interfaz gráfica de usuario

En esta sección describiremos las distintas clases de nuestra GUI.

Ventana principal

La ventana principal está representada por la siguiente clase. Lee el código y completa las partes que no están implementadas. En lugar de `BoxLayout` se puede usar `GridBagLayout` o `GridLayout`.

```
public class MainWindow extends JFrame {

    private Controller ctrl;

    public MainWindow(Controller ctrl) {
        super("[ECOSYSTEM SIMULATOR]");
        this.ctrl = ctrl;
        initGUI();
    }

    private void initGUI() {
        JPanel mainPanel = new JPanel(new BorderLayout());
        setContentPane(mainPanel);

        // TODO crear ControlPanel y añadirlo en PAGE_START de mainPanel

        // TODO crear StatusBar y añadirlo en PAGE_END de mainPanel

        // Definición del panel de tablas (usa un BoxLayout vertical)
        JPanel contentPanel = new JPanel();
        contentPanel.setLayout(new BoxLayout(contentPanel, BoxLayout.Y_AXIS));
        mainPanel.add(contentPanel, BorderLayout.CENTER);
    }
}
```

```

// TODO crear la tabla de especies y añadirla a contentPanel.
//      Usa setPreferredSize(new Dimension(500, 250)) para fijar su tamaño

// TODO crear la tabla de regiones.
//      Usa setPreferredSize(new Dimension(500, 250)) para fijar su tamaño

// TODO llama a ViewUtils.quit(MainWindow.this) en el método windowClosing
addWindowListener(...);

setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
pack();
setVisible(true);
}
}

```

Barra de control

El panel de control es el responsable de la interacción entre el usuario y el simulador. Se corresponde con la barra de herramientas que aparece en la parte superior de la ventana (ver el apartado Figuras).

Incluye los siguientes componentes: botones para interactuar con el simulador, un `JSpinner` para seleccionar los pasos de simulación deseados, y un `JTextField` para actualizar el delta-time. El valor inicial que tiene que aparecer en el delta-time es el del atributo correspondiente en la clase `Main`.

```

class ControlPanel extends JPanel {

    private Controller ctrl;
    private ChangeRegionsDialog changeRegionsDialog;

    private JToolBar toolBar;
    private JFileChooser fc;
    private boolean stopped = true; // utilizado en los botones de run/stop
    private JButton quitButton;

    // TODO añade más atributos aquí ...

    ControlPanel(Controller ctrl) {
        this.ctrl = ctrl;
        initGUI();
    }

    private void initGUI() {
        setLayout(new BorderLayout());
        toolaBar = new JToolBar();
        add(toolaBar, BorderLayout.PAGE_START);
    }
}

```

```

// TODO crear los diferentes botones/atributos y añadirlos a la toolBar.
//     Todos ellos han de tener su correspondiente tooltip. Puedes utilizar
//     this.toolaBar.addSeparator() para añadir la línea de separación vertical
//     entre las componentes que lo necesiten.

// Quit Button
this.toolaBar.add(Box.createGlue()); // this aligns the button to the right
this.toolaBar.addSeparator();
this.quitButton = new JButton();
this.quitButton.setToolTipText("Quit");
// TODO cargar la imagen como un recurso usando el ClassLoader y NO usando una ruta abs
this.quitButton.setIcon(new ImageIcon("..."));
this.quitButton.addActionListener((e) -> ViewUtils.quit(this));
this.toolaBar.add(quitButton);

// TODO Inicializar this.fc con una instancia de JFileChooser. Para que siempre
// abre en la carpeta de ejemplos puedes usar:
//
//     this.fc.setCurrentDirectory(new File(System.getProperty("user.dir") + "/resources/

// TODO Inicializar this.changeRegionsDialog con instancias del diálogo de cambio
// de regiones

}
// TODO el resto de métodos van aquí...
}

```

La funcionalidad de los distintos botones es la siguiente:

- Cuando se pulsa el botón: (1) utiliza `this.fc.showOpenDialog(ViewUtils.getWindow(this))` para abrir el selector de ficheros para que el usuario pueda seleccionar el archivo de entrada; (2) si el usuario ha seleccionado un fichero, cárgalo como `JSONObject`, resetea el simulador utilizando `this.ctrl.reset(...)` con parámetros correspondiente, y carga el json usando `this.ctrl.loadData(...)`.
- Cuando se pulsa el botón crea una instancia de `MapWindow` (descripción a continuación). Esto permite al usuario ver una representación visual de la simulación. Ten en cuenta que el usuario puede tener varios visores abiertos al mismo tiempo.
- Cuando se pulsa el botón llama a `this.changeRegionsDialog.open(ViewUtils.getWindow(this))` para abrir el diálogo de regiones (recuerda que la instancia se crea sólo una vez en la constructora).
- Cuando se pulsa el (1) deshabilita todos los botones excepto el botón de stop, y cambia el valor del atributo `this.stopped` a `false`; (2) saca el

valor del delta-time del correspondiente `TextField`; y (3) llama al método `runSim` con el valor actual de pasos, especificado en el correspondiente `Spinner`:

```
private void runSim(int n, double dt) {
    if (n > 0 && !this.stopped) {
        try {
            this.ctrl.advance(dt);
            SwingUtilities.invokeLater(() -> runSim(n - 1, dt));
        } catch (Exception e) {
            // TODO llamar a ViewUtils.showErrorMsg con el mensaje de error
            //      que corresponda
            // TODO activar todos los botones
            this.stopped = true;
        }
    } else {
        // TODO activar todos los botones
        this.stopped = true;
    }
}
```

Debes completar el método `runSim` como se indica en los comentarios. Fíjate que el método `runSim` tal y como está definido garantiza que el interfaz no se quedará bloqueado. Para entender este comportamiento modifica `runSim` para incluir solo `for(int i=0; i<n; i++)this.ctrl.advance(dt)` — ahora, al comenzar la simulación, no verás los pasos intermedios, únicamente el estado final, además de que la interfaz estará completamente bloqueada.

- Cuando se pulsa el botón , actualiza el valor del atributo `this.stopped` a `true`. Esto “detendrá” el método `runSim` si hay llamadas en la cola de eventos de swing (observa la condición del método `runSim`).
- La funcionalidad del se proporciona como parte del código.

[!Important] Debes capturar todas las posibles excepciones lanzadas por el controlador/simulador y mostrar el correspondiente mensaje utilizando `ViewUtils.showErrorMsg`. No escribas errores con `System.out` o `System.err`, ni con `stackTrace()` de la excepción.

Barra de estado

La barra de estado es la responsable de mostrar información general sobre el simulador. Se corresponde con el área de la parte inferior de la ventana (ver el apartado Figuras). Lee el código y completa las partes que faltan. Es obligatorio añadir el tiempo de simulación, el número total de animales, y la dimensión de la simulación (anchura, altura, filas, y columnas). Puedes añadir más información si lo deseas. Actualiza los distintos valores desde los métodos de

EcoSysObserver cuando sea necesario.

```
class StatusBar extends JPanel implements EcoSysObserver {

    // TODO Añadir los atributos necesarios.

    StatusBar(Controller ctrl) {
        initGUI();
        // TODO registrar this como observador
    }

    private void initGUI() {
        this.setLayout(new FlowLayout(FlowLayout.LEFT));
        this.setBorder(BorderFactory.createBevelBorder(1));

        // TODO Crear varios JLabel para el tiempo, el número de animales, y la
        //       dimensión y añadirlos al panel. Puedes utilizar el siguiente código
        //       para añadir un separador vertical:
        //
        //       JSeparator s = new JSeparator(JSeparator.VERTICAL);
        //       s.setPreferredSize(new Dimension(10, 20));
        //       this.add(s);
    }

    // TODO el resto de métodos van aquí...
}
```

Tablas de Información

Las tablas son las responsables de mostrar la información de los animales/regiones. Tendremos una clase `InfoTable` que incluye un `JTable` y que recibirá como parámetro el correspondiente modelo de la tabla. Para ello tendremos también dos clases `SpeciesTableModel` y `RegionsTableModel` para los modelos de tablas de las especies y de las regiones, respectivamente.

Como las tablas tienen partes comunes, vamos a definir una clase que representa una tabla que recibe el modelo de tabla (que incluye los datos) como parámetro y usarla para ambas tablas:

```
public class InfoTable extends JPanel {

    private String title;
    private TableModel tableModel;

    InfoTable(String title, TableModel tableModel) {
        this.title = title;
        this.tableModel = tableModel;
    }
}
```

```

        initGUI();
    }

    private void initGUI() {
        // TODO cambiar el layout del panel a BorderLayout()
        // TODO añadir un borde con título al JPanel, con el texto this.title
        // TODO añadir un JTable (con barra de desplazamiento vertical) que use
        //      this.tableModel
    }
}

```

Usando `InfoTable`, la creación de las tablas en `MainWindow` se puede implementar así:

```

new InfoTable("Species", new SpeciesTableModel(this.ctrl));
new InfoTable("Regions", new RegionsTableModel(this.ctrl));

```

donde `SpeciesTableModel` y `RegionsTableModel` están descritas a continuación.

Tabla de especies

El primer modelo de tabla representa la tabla de especies y será representado por la siguiente clase:

```

class SpeciesTableModel extends AbstractTableModel implements EcoSysObserver {

    // TODO definir atributos necesarios

    SpeciesTableModel(Controllor ctrl) {
        // TODO inicializar estructuras de datos correspondientes
        // TODO registrar this como observador
    }
    // TODO el resto de métodos van aquí ...
}

```

La tabla incluye una fila para cada código genético con información sobre el número de animales en cada posible estado (ver el apartado Figuras).

[!IMPORTANT] Si añadimos más códigos genéticos y/o estados al simulador, la tabla tiene que seguir funcionando igual sin la necesidad de modificar nada de su código, y por eso (1) está prohibido hacer referencia explícita a códigos genéticos como "sheep" y "wolf", esta información hay que sacarla de la lista de animales; (2) está prohibido hacer referencia a estados concretos como `NORMAL`, `DEAD`, etc. Hay que usar `State.values()` para saber cuáles son los posibles estados.

Tabla de regiones

El segundo modelo de tabla representa la tabla de regiones y será representado por la siguiente clase:

```
class RegionsTableModel extends AbstractTableModel implements EcoSysObserver {

    // TODO definir atributos necesarios

    RegionsTableModel(Controller ctrl) {
        // TODO inicializar estructuras de datos correspondientes
        // TODO registrar this como observador
    }
    // TODO el resto de métodos van aquí...
}
```

La tabla incluye una fila para cada región con información sobre su fila y columna en la matriz de regiones, su descripción (lo que devuelve `toString()` de la región), y el número de animales en la región para cada tipo de dieta (ver el apartado Figuras).

[!IMPORTANT] Si añadimos más tipos de dietas al simulador la tabla tiene que seguir funcionando igual sin la necesidad de modificar nada de su código, y por eso está prohibido hacer referencia explícita a tipos de dietas como `CARNIVORE` y `HERBIVORE`. Hay que usar `Diet.values()` para saber cuáles son las posibles dietas.

Diálogo de cambio de regiones

La clase `ChangeRegionsDialog` es la responsable de implementar la ventana de diálogo que permite modificar las regiones (ver el apartado Figuras):

```
class ChangeRegionsDialog extends JDialog implements EcoSysObserver {

    private DefaultComboBoxModel<String> regionsModel;
    private DefaultComboBoxModel<String> fromRowModel;
    private DefaultComboBoxModel<String> toRowModel;
    private DefaultComboBoxModel<String> fromColModel;
    private DefaultComboBoxModel<String> toColModel;

    private DefaultTableModel dataTableModel;
    private Controller ctrl;
    private List<JSONObject> regionsInfo;

    private String[] headers = { "Key", "Value", "Description" };

    // TODO en caso de ser necesario, añadir los atributos aquí...
    ChangeRegionsDialog(Controller ctrl) {
```

```

        super((Frame)null, true);
        this.ctrl = ctrl;
        initGUI();
        // TODO registrar this como observer;
    }

    private void initGUI() {
        setTitle("Change Regions");
        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
        setContentPane(mainPanel);

        // TODO crea varios paneles para organizar los componentes visuales en el
        //      dialogo, y añadelos al mainpanel. P.ej., uno para el texto de ayuda,
        //      uno para la tabla, uno para los combobox, y uno para los botones.

        // TODO crear el texto de ayuda que aparece en la parte superior del diálogo y
        //      añadirlo al panel correspondiente diálogo (Ver el apartado Figuras)

        // this.regionsInfo se usará para establecer la información en la tabla
        this.regionsInfo = Main.regionsFactory.getInfo();

        // this.dataTableModel es un modelo de tabla que incluye todos los parámetros de
        // la region
        this.dataTableModel = new DefaultTableModel() {
            @Override
            public boolean isCellEditable(int row, int column) {
                // TODO hacer editable solo la columna 1
            }
        };
        this.dataTableModel.setColumnIdentifiers(this.headers);

        // TODO crear un JTable que use dataTableModel, y añadirlo al diálogo

        // this.regionsModel es un modelo de combobox que incluye los tipos de regiones
        this.regionsModel = new DefaultComboBoxModel<>();

        // TODO añadir la descripción de todas las regiones a regionsModel. Para eso
        //      usa la clave "desc" o "type" de los JSONObject en regionsInfo,
        //      ya que estos nos dan información sobre lo que puede crear la factoría.

        // TODO crear un combobox que use regionsModel y añadirlo al diálogo.

        // TODO crear 4 modelos de combobox para this.fromRowModel, this.toRowModel,
        //      this.fromColModel y this.toColModel.

```

```

// TODO crear 4 combobox que usen estos modelos y añadirlos al diálogo.

// TODO crear los botones OK y Cancel y añadirlos al diálogo.

setPreferredSize(new Dimension(700, 400)); // puedes usar otro tamaño
pack();
setResizable(false);
setVisible(false);
}

public void open(Frame parent) {
    setLocation(
        parent.getLocation().x + parent.getWidth() / 2 - getWidth() / 2,
        parent.getLocation().y + parent.getHeight() / 2 - getHeight() / 2);
    pack();
    setVisible(true);
}

// TODO el resto de métodos van aquí...
}

```

El diálogo se crea/abre en **ControlPanel** cuando se pulsa sobre el correspondiente botón. Recuerda que se debe crear una única instancia de la ventana de diálogo en la constructora, y después basta con llamar al método **open**. De esta forma el diálogo mantendrá su último estado. La funcionalidad a implementar es la siguiente:

1. En los métodos **onReset** y **onRegister** de **EcoSysObserver** debes mantener la lista de opciones en los combobox de coordenadas actualizada – usa **removeAllElements** y **addElement** del modelo correspondiente. Así cuando cambia el número de fila/columnas cambian también en los combobox.
2. Cuando el usuario selecciona la *i*-ésima región (del correspondiente combobox), debes actualizar **this.dataTableModel** para tener las claves y las descripciones en la primera y tercera columna respectivamente, lo que modificará el contenido de la correspondiente **JTable**. Para implementar este comportamiento (a) obtén el *i*-ésimo elemento de **this.regionsInfo**, llámalo **info**; (b) obtén el valor asociado a la clave "**data**" de **info**, llámalo **data**; y (3) itera sobre **data.keySet()** y añade cada elemento a la primera columna y su valor (que es la descripción) en la tercera columna.
3. Si el usuario pulsa el botón Cancel, simplemente pon **this.status** a 0 y haz el diálogo invisible.
4. Si el usuario pulsa el botón OK
 1. Convierte la información en la tabla en un JSON que incluye la clave y el valor para cada fila en la tabla, sólo para la fila que incluyen valor no vacío – en el ejemplo **extra.dialog.ex3** hay un método que hace algo parecido. Nos referimos a este JSON como **region_data**.

2. Saca el tipo de la región seleccionada usando el índice seleccionado; puedes hacerlo desde `this.regionsInfo`. Nos referimos a este valor como `region_type`.
3. Saca las coordenadas de los combobox correspondientes. Nos referimos a estos valores como `row_from`, `row_to`, `col_from`, `col_to`.
4. Crea un JSON de la forma:

```
{
  "regions" : [ {
    "row" : [ row_from, row_to ],
    "col" : [ col_from, col_to ],
    "spec" : {
      "type" : region_type,
      "data" : region_data
    }
  }
]
```

y pasalo a `this.ctrl.setRegions` para cambiar las regiones. Si la llamada acaba con éxito, pon `this.status` a 1 y haz el diálogo invisible; en otro caso, muestra el mensaje de la excepción correspondiente usando `ViewUtils.showErrorMsg`. No escribas errores con `System.out` o `System.err`, ni con `stackTrace()` de la excepción.

[!IMPORTANT] Si añadimos más tipos de regiones a la factoría de regiones, el diálogo tiene que seguir funcionando igual sin la necesidad de modificar nada de su código, y por eso está prohibido hacer referencia explícita a tipos de regiones como `"default"` y `"dynamic"`, ni a claves como `"factor"` y `"food"`. Siempre hay que sacar la información usando `getInfo()` de la factoría.

Visor del mapa

Este componente dibuja el estado de la simulación gráficamente en cada paso (Ver el apartado Figuras). Es implementado por dos clases: una clase llamada `MapWindow` que representa la ventana, y una clase llamada `MapView` que hace la visualización (extiende una clase abstracta llamada `AbstractMapView` de tal forma que nos abstraemos de la implementación actual; notar que `AbstractMapView` extiende `JComponent` así que podemos tratar una instancia como un componente Swing). Lo siguiente es un esqueleto de `MapWindow`:

```
class MapWindow extends JFrame implements EcoSysObserver {

    private Controller ctrl;
    private AbstractMapView viewer;
    private Frame parent;

    MapWindow(Frame parent, Controller ctrl) {
```

```

    super("[MAP VIEWER]");
    this.ctrl = ctrl;
    this.parent = parent;
    intiGUI();
    // TODO registrar this como observador
}

private void intiGUI() {
    JPanel mainPanel = new JPanel(new BorderLayout());
    // TODO poner contentPane como mainPanel

    // TODO crear el viewer y añadirlo a mainPanel (en el centro)

    // TODO en el método windowClosing, eliminar 'MapWindow.this' de los
    // observadores
    addWindowListener(new WindowListener() { ... });

    pack();
    if (this.parent != null)
        setLocation(
            this.parent.getLocation().x + parent.getWidth() / 2 - getWidth() / 2,
            this.parent.getLocation().y + parent.getHeight() / 2 - getHeight() / 2);
    setResizable(false);
    setVisible(true);
}
// TODO otros métodos van aquí...
}

```

Nótese que la ventana no se puede redimensionar para que el código que dibuje el estado en `this.viewer` sea más sencillo.

Deberías completar el código de los métodos de `EcoSysObserver` de forma que:

1. Los métodos `onRegister` y `onReset` llamen al `reset` del `this.viewer` y cambien el tamaño de la ventana usando `pack()` porque el `this.viewer` puede cambiar de tamaño. Esto se puede hacer usando `SwingUtilities.invokeLater(() -> { this.viewer.reset(...); pack(); });`
2. El método `onAdvance` llame a `update` del `this.viewer`. Esto se puede hacer usando `SwingUtilities.invokeLater(() -> { this.viewer.update(...) });`

La clase `MapView.java` y `AbstractMapViwer.java` son dadas sin parte de su funcionalidad. Lee todos los comentarios **TODO** dentro del código y complétalos – se dará más información en las clases/laboratorios. En general el visor tiene que: (1) dibujar cada animal con un tamaño relativo a su edad y de color que corresponde a su código genético; (2) mostrar información sobre el tiempo actual y el número de animales de cada código genético; y (3) permitir mostrar

solo animales que tienen estado específico (pulsando la tecla **s** cambiamos de un estado a otro).

[!IMPORTANT] Si añadimos más códigos genéticos al simulador, este componente tiene que seguir funcionando igual sin la necesidad de modificar nada de su código, y por eso está prohibido hacer referencia explícita a códigos genéticos como “sheep” y “wolf”, esta información hay que sacarla de la lista de animales.

Cambios en la clase Main

Nueva opción `--mode`

En la clase `Main` es necesario añadir una nueva opción `-m` que permita al usuario usar el simulador en modo **BATCH** (como en la Práctica 1) y en modo **GUI**. Esta opción es opcional con un valor predeterminado que inicia el modo **GUI**:

```
usage: simulator.launcher.Main [-dt <arg>] [-h] [-i <arg>] [-m <arg>] [-o
    <arg>] [-sv] [-t <arg>]

-dt,--delta-time <arg>  A real number representing actual time, in
                           seconds, per simulation step. Default value:
                           0.03.
-h,--help                Print this message.
-i,--input <arg>         A configuration file (optional in GUI mode).
-m,--mode <arg>          Execution Mode. Possible values: 'batch' (Batch
                           mode), 'gui' (Graphical User Interface mode).
                           Default value: 'gui'.
-o,--output <arg>        A file where output is written (only for BATCH mode).
-sv,--simple-viewer       Show the viewer window in BATCH mode.
-t,--time <arg>          An real number representing the total simulation
                           time in seconds. Default value: 10.0. (only for BATCH mode).
```

Dependiendo del valor dado para la opción `-m`, el método `start` invoca al método `startBatchMode` o al nuevo método `startGUIMode`. Ten en cuenta que a diferencia del modo **BATCH**, en el modo **GUI** el parámetro `-i` es opcional. Las opciones `-o` y `-t` se ignoran en el modo **GUI**. Recuerda que las opciones `-i` y `-o` tienen que seguir siendo obligatorias en el modo **BATCH**.

Método `startGUIMode`

Completa el método `startGUIMode` de manera parecida a `startBatchMode`, pero sin llamar al método `run` del controlador sino creando una ventana usando:

```
SwingUtilities.invokeLaterAndWait(() -> new MainWindow(ctrl));
```

Recuerda que si el usuario proporciona un archivo de entrada hay que usarlo para crear la instancia de `Simulator` y además añadir los animales y regiones

Ventana principal

Figure 1: Ventana principal

Cambio de regiones

Figure 2: Cambio de regiones

usando `loadData` del controlador. Si no lo proporciona crea la instancia de `Simulator` con valores por defecto para la anchura, altura, filas y columnas (se puede usar 800, 600, 15, 20). Recuerda que no hay que usar archivo de salida en este modo.

Figuras

Ventana principal

Diálogo de cambio de regiones

Visor del mapa

Exportar el proyecto a un JAR

Este apartado no forma parte de la práctica. Es para practicar cómo exportar el proyecto a un **JAR** y ejecutarlo desde la línea de comandos.

Los iconos

Los iconos forman parte de la aplicación, por eso decimos que son un *recurso*, y los guardamos en `src/main/resources/icons`. Para garantizar que esos iconos están siempre disponibles, incluso cuando “empaquetemos” nuestro código en un JAR, debemos cargar este recurso de una forma concreta. Tenéis más detalle en las transparencias del tema de test unitarios.

La ventaja de tener un proyecto Maven es que estando los recursos en los directorios apropiados, al compilarse y generarse el JAR tendremos los recursos disponibles para usarlos en tiempo de ejecución.

Exportar a un JAR

Ya sabemos que al compilar un proyecto Java, por lo general, el resultado es un fichero JAR que contiene todos los ficheros `.class` listos para ser utilizados por la máquina virtual de Java.

Visor del mapa

Figure 3: Visor del mapa

Existe un tipo especial de JAR que es el denominado *JAR ejecutable* (en inglés, *runnable JAR*): este JAR se puede ejecutar como un programa habitual de nuestro PC, simplemente haciendo doble *click* sobre él.

Vamos a hacer que el JAR de nuestro proyecto sea ejecutable. Bueno, nosotros no: vamos a indicar a Maven, que para eso trabaja para nosotros, que cuando genere el JAR de la aplicación, este sea ejecutable.

Debemos tener en cuenta una cosa importante: si queremos que el JAR sea ejecutable, necesitamos que todas las 4 dependencias que necesita nuestra aplicación estén disponibles también. Esto requiere las librerías

- `org.json:json`
- `commons-cli:commons-cli`

que actualmente Maven descarga de internet y disponibiliza por nosotros, estén incluidas en el JAR ejecutable.

Estos JAR que a su vez contienen sus dependencias se denominan *fat JAR*, y están pensados para hacer más sencillo “portar” las aplicaciones. Si todas las dependencias están en un único fichero, no se necesita ningún recurso adicional donde vayamos a ejecutar nuestro JAR *porque todo lo necesario el propio JAR lo tiene incluido*. Dicho esto, nuestro objetivo es generar ahora un *fat JAR ejecutable*.

Esto se puede hacer empleando un *plugin* de Maven llamado `maven-assembly-plugin`. Podemos añadirlo y configurarlo en la sección de plugins de nuestro `pom.xml`:

```
<plugin>
  <artifactId>maven-assembly-plugin</artifactId>
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>single</goal>
      </goals>
      <configuration>
        <!-- this block tells assembly plugin to build an executable JAR -->
        <archive>
          <manifest>
            <mainClass>
              simulator.launcher.Main
            </mainClass>
          </manifest>
        </archive>
        <!-- this configuration tells assembly plugin to build a fat JAR -->
        <descriptorRefs>
          <descriptorRef>jar-with-dependencies</descriptorRef>
        </descriptorRefs>
      </configuration>
    </execution>
  </executions>
</plugin>
```

```

        </configuration>
    </execution>
</executions>
</plugin>

```

- Para generar un jar ejecutable, debemos indicar cuál es la clase principal. Esto lo hacemos configurando la sección **manifest**
- Para generar un fatjar, indicamos al plugin que use una configuración de referencia (que ya viene incluida en el plugin) y que se llama **jar-with-dependencies**

Una vez configurado, si lanzamos Maven con la “fase” **package**, veremos que se generan dos JARs dentro de la carpeta **target/**:

```

-rw-r--r-- 1 235K feb. 25 target/ecosystem-simulator-1.0.0-SNAPSHOT-jar-with-dependencies.jar
-rw-r--r-- 1 123K feb. 25 target/ecosystem-simulator-1.0.0-SNAPSHOT.jar

```

Queda bastante claro cuál de los dos es el fatjar y a qué debe su nombre.

Ejecutar desde la línea de comandos

El fatjar se puede ejecutar directamente en el terminal de la siguiente manera:

```

java -jar target/ecosystem-simulator-1.0.0-SNAPSHOT-jar-with-dependencies.jar -h
usage: simulator.launcher.Main [-dt <arg>] [-h] [-i <arg>] [-m <arg>] [-o
    <arg>] [-sv] [-t <arg>]
    -dt,--delta-time <arg>    A real number representing actual time, in
                                seconds, per simulation step. Default value:
                                0.03.
    -h,--help                  Print this message.
    -i,--input <arg>           A configuration file.
    -m,--mode <arg>            Execution Mode. Possible values: 'batch' (Batch
                                mode), 'gui' (Graphical User Interface mode).
                                Default value: 'gui'.
    -o,--output <arg>          A file where output is written.
    -sv,--simple-viewer         Show the viewer window in console mode.
    -t,--time <arg>            An real number representing the total simulation
                                time in seconds. Default value: 10.0.

```

Otra opción para ejecutarlo es usando el plugin **exec-maven-plugin**:

```

mvn exec:java "-Dexec.args=-h"

```